

Chess Bot Project: A Tune-Up

Finishing Phase 4 the right way

for Imran

Imran, the hard part is already behind you! Your move generation is correct: I ran perft (count every leaf of the move tree and compare against a known-correct engine) on the standard test positions, including Kiwipete, the one built specifically to break castling and en passant, and your counts matched to the digit. Castling, en passant, promotion, make and unmake: all of it works, and every board test passes. You also tracked down and killed the file-wrap bug in `is_attacked` that was getting you flagged in the arena; I reran that exact position and your `legal_moves()` now matches the reference. Getting the board exactly right is the part of this project that breaks the most people, and you are past it.

So this is a tune-up, not a repair job. There is one real bug left in what you've built, the one your last commit calls "a bug," and the rest is about making your evaluator a little stronger and your code a little cleaner before search arrives in Phase 5. Each cleanup below works the same way: a symptom you can run and see, a short hunt to where it lives in your code, why it happens, and a fix you write. I'll give you the measuring scripts, but the fixes are yours.

Setup. Work in your `chess-bot` folder with your virtual environment active:

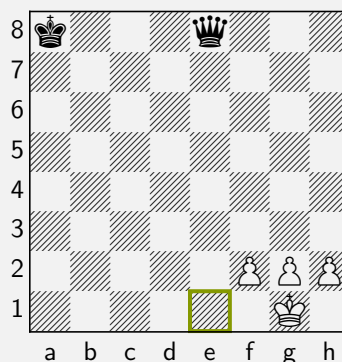
```
cd chess-bot
source .venv/bin/activate
```

Save each measuring script (the name is given each time) into that folder and run it with `python <name>.py`; they import `board`, `evaluation`, and `bot` the way your tests do. Keep `board.py`, `evaluation.py`, and `bot.py` open.

Cleanup 1: The Bot That Helps Its Opponent

Your move-picker wins as White and throws the game as Black

Here is the position your one failing test is built on. Black is to move, and Black has a mate in one: `Qe8-e1#`.



Save this as `pick.py` and run it:

```
1 from board import Board
2 from bot import choose_move
3
4 # Black to move; Qe8-e1 is checkmate.
5 board = Board.from_fen("k3q3/8/8/8/8/8/5PPP/6K1 b - - 0 1")
6 print("the mate is e8e1; your bot plays:", choose_move(board, 1000).uci())
```

Your bot plays `e8h8`, a quiet shuffle of the queen into the corner, strolling past a checkmate it is allowed to play. And it isn't picky about this one position; given any winning move as Black, it will do the same thing. This is the bug your commit message means, and `chess-cli test -k stage14` fails on exactly this case.

Step 1.

Pin down what the bot is actually optimizing. Open `bot.py` and read `choose_move`. It scores every legal move by making it and calling `evaluate` on the result, then returns the move with the highest score (`bot.py:28`, the `max(moves)`). Now remind yourself what `evaluate` measures: read its docstring. The score is from *White's* point of view, positive for White, negative for Black. With that in mind, answer one question on paper: when it is Black to move, does Black want the score after its move to be large or small?

Step 2.

Now you can see the bug. `evaluate` after `Qe8-e1#` returns about $-1,000,000$, because White has been checkmated and that is wonderful for Black. But your bot takes the *maximum* score, so it discards the most negative move, the mate, and reaches for whatever helps White most. Taking the maximum is correct when you are White (White wants the biggest White-favoring number); it is exactly backwards when you are Black, who wants the most negative score it can get. Your bot, playing Black, is choosing the move that is best for its opponent.

Step 3.

Fix `choose_move` so it picks the best move *for the side to move*, which is the whole point of the Stage 14 “eval-and-pick” idea. You have two clean ways to do it: branch on `board.side_to_move` and take the `max` when it is White's turn but the `min` when it is Black's; or flip the sign of the score for Black so that “bigger is always better for me” and keep a single `max`. Either is fine. Then re-run `pick.py` (the bot should now play `e8e1`) and `chess-cli test -k stage14` (green). That was the last red test in Phase 4.

Cleanup 2: A Mobility Term With One Eye Closed

Counting only your own moves, and counting them four times

Your evaluator passes its tests, so this cleanup is about making it both *more correct* and *much cheaper*, two separate problems that happen to live in the same few lines.

Start with correctness. A static evaluation is supposed to be one number for a position, the same number no matter whose clock is running. Check whether yours is. Save as `sidedep.py`:

```
1 from board import Board
2 from evaluation import evaluate
3
4 fen = "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R w KQkq - 2 3"
5 board = Board.from_fen(fen)
6 print("eval, White to move:", evaluate(board))
7 board.state.side_to_move = board.state.side_to_move.other # flip ONLY the turn
8 print("eval, Black to move:", evaluate(board))
```

It prints 54 and then -62: the same pieces on the same squares, scored 116 centipawns apart, with nothing changed but the side-to-move flag. A static score should not move at all.

Step 1.

Find the cause in `evaluation.py:121--124`:

```
if board.side_to_move == Color.WHITE:
    score += len(board.legal_moves()) * 2
else:
    score -= len(board.legal_moves()) * 2
```

This adds the mobility of *whoever is to move* and never looks at the other side, so flipping the turn flips which side's move count gets counted, which is the 116-point swing. The Stage 16 handout asks for something different: *White's* mobility minus *Black's*, always, regardless of turn. That single quantity does not depend on whose move it is, and it uses both halves of the information instead of throwing one away.

Step 2.

Rewrite the term as `(white_move_count - black_move_count) * weight`. The wrinkle is that `legal_moves()` only ever gives you the side to move's moves. To get the other side's count, do what the kit's own evaluator does: temporarily set `board.state.side_to_move` to the other color, call `legal_moves()`, and set it back, so the board ends exactly as it started. Re-run `sidedep.py` and confirm the two numbers are now equal.

Step 3.

Now the cheap-it-up half. Save as `count.py`:

```
1 from board import Board
2 import evaluation
3
4 calls = 0
```

```

5 original = Board.legal_moves
6 def counting(self):
7     global calls
8     calls += 1
9     return original(self)
10
11 board = Board.from_fen(
12     "r1bqkb1r/pppp1ppp/2n2n2/4p3/4P3/2N2N2/PPPP1PPP/R1BQKB1R w KQkq - 4 4")
13 Board.legal_moves = counting
14 evaluation.evaluate(board) # one single evaluation
15 Board.legal_moves = original
16 print("legal_moves() calls in ONE evaluate():", calls)

```

It prints 4. `legal_moves()` is the most expensive thing in your engine (it makes and unmakes every move to test for check), and one evaluation calls it four times: once in each of your three terminal-position checks (`evaluation.py:55, 62, 69`) and once more for mobility. The three terminal checks and your own side's move count can all share one list: call `legal_moves()` once at the top of `evaluate`, store it, and use `if not legal:` for the terminal cases and `len(legal)` for your side's half of the mobility term. The opponent's half (the side-flip you added in Step 2) genuinely needs its own call, so `count.py` should drop from four to *two*, not to one; two is the real floor here, and it is half of what you were paying. This barely matters today, but in Phase 5 your search will call `evaluate` thousands of times for a single move, and a four-times-too-slow evaluator is a four-times-slower bot.

Cleanup 3: Loose Ends

Things that pretty much work, but still smell bad

A short grab-bag. None of these fails a test, just a place where the code gets the right answer in a slightly fragile or roundabout way. Fix as many of these as you can.

Step 1.

`castling_moves` scribbles on a shared class (`board.py:532--535`). Those four lines copy each `self.state.castling` flag onto an attribute of the imported `CastlingRights` class itself, which every board in the program shares, and then read it straight back one line later. It works only because you read exactly what you just wrote. There is no reason to route through the class at all: read the `self.state.castling` flags directly in each `if`, and delete those four lines.

Step 2.

`make_move` and `undo_move` deep-copy pieces (`board.py:296, 307, 414, and friends`). `copy.deepcopy(self.pieces[...])` does a lot of work to relocate a piece you could move with a plain assignment, `self.pieces[to] = self.pieces[from]`. You never change a `Piece` in place after putting it down, so the deep copy protects against nothing and costs real time on every single move, which perft and (soon) search make millions of. Drop the `deepcopy` calls for direct assignment.

Step 3.

`pawn_attacking_square` guards the edge in the wrong order (`board.py:473--494`). Each of those lines tests `piece_at(sq(...))` *before* it tests `on_board(...)`: you compute a square that may have wrapped across the board edge and look at it before you have confirmed it is on the board. It comes out correct only because `piece_at` guards truly off-board indices and the `and` stops at the `on_board` half, a chain of luck you should not have to rely on. Put the `on_board` test first. While you are in there, the condition `p.color == WHITE` and `p.color == color` is just `color == WHITE` written the long way; the function gets much shorter if you work out the two squares a `by_color` pawn would attack from and check only those.

Step 4.

A couple of quick ones. A leftover `move()` helper sits unused at the top of `board.py` (lines 49–52); if it was a debugging aid, delete it. And both `board.py` and `evaluation.py` import a pile of names they never use (`evaluation.py`, for instance, imports `MoveRecord`, `KING_OFFSETS`, `parse_square`, and more that it never touches); trimming them makes it clear at a glance what each file actually needs.

Summit *Prove it plays both colors, then look ahead*

With Cleanup 1 in, your bot no longer sabotages itself as Black. Prove it, and then notice what you have actually built.

The proof. Confirm the three things the cleanups touched, with filters chosen so they don't accidentally sweep in the Phase-5 search tests:

```
chess-cli test -k stage14                # was red, now green (Cleanup 1)
chess-cli test -k "stage13 or stage15 or stage16" # evaluator still sound (Cleanup
2)
chess-cli perft 3                          # board intact: 8902, matching
the reference (Cleanup 3)
```

The Stage 14 mate case that was red is the one to watch. Leave the Phase-5 search tests (`stage17` through `stage20`) out of every filter: `search.py` is still a stub, so they are meant to fail until you reach Phase 5, and a bare term like `-k stage1` would quietly pull `stage17–stage19` in alongside the board tests. Then set up a position where *White* has a mate in one and confirm the bot finds it too, so you know the fix works in both directions rather than just flipping the bug to the other color.

The look-ahead. Step back and see what `choose_move` is now doing: it tries every move, scores the result with `evaluate`, and keeps the best one for the side to move. That is a search exactly one move deep, and the `max-for-White`, `min-for-Black` rule you just wrote by hand is the whole idea of minimax, the heart of Phase 5. There, instead of scoring the position right after your move, you will let the opponent reply with *their* best move, and reply to that, looking several moves ahead before the evaluator is ever consulted; the maximizing and minimizing you just sorted out is what alternates down that tree. Your one-ply picker is the base case of the

thing you are about to build, which means the cleaner and more correct you make **evaluate** now, the better every leaf of that future search will be.

Checkpoint. Your board was already right, and now your bot plays for the side whose turn it actually is, so it stops handing wins to its opponent every time it has the black pieces. Your evaluator gives one honest number per position instead of a different one each turn, and it does so without generating the move list four times over. The smaller tangles are straightened out, so the next time you open this code it says what it means. That is a finished Phase 4: a correct board, a sound evaluator, and a bot that picks the right move one ply deep. Phase 5 takes that exact picker and teaches it to look further than one move, which is where the chess starts to get good.